

CPSC 375 High-Performance Computing
Lecture 27

Performance Analysis

Speedup

- Number of processes = p
- Serial run-time = T_s
- Parallel run-time = T_p

$$T_p = T_s / p$$

Linear speedup

Speedup

- In practice, we're unlikely to get linear speedup because the use of multiple processes almost invariably introduces some overhead.
 - e.g., programs will almost always need to transmit data across the network, which is usually much slower than local memory access.
- It's likely that the overheads will increase as we increase the number of processes.
 - More processes will probably mean more data needs to be transmitted across the network.

Speedup of a parallel program

- Define

$$S_p = \frac{T_s}{T_p}$$

- *Linear Speedup*: $S = p$. This is the "Gold Standard" but, is very unusual in practice.
- *Sub-linear Speedup*: $S < p$. As you increase the number of processes (p), the speedup S typically becomes a smaller and smaller fraction of the ideal.

Efficiency of a parallel program

$$E = \frac{S_p}{p} = \frac{\left(\frac{T_s}{T_p} \right)}{p} = \frac{T_s}{p \cdot T_p}$$

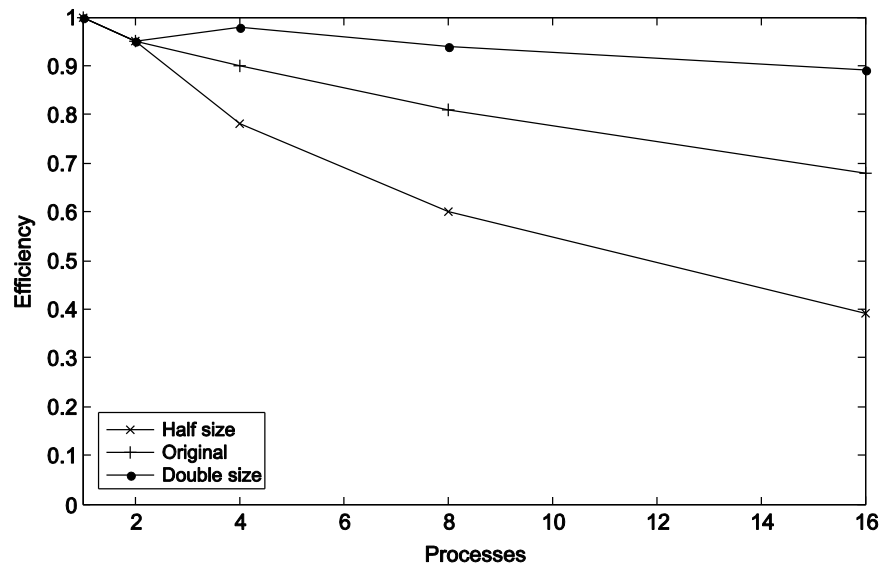
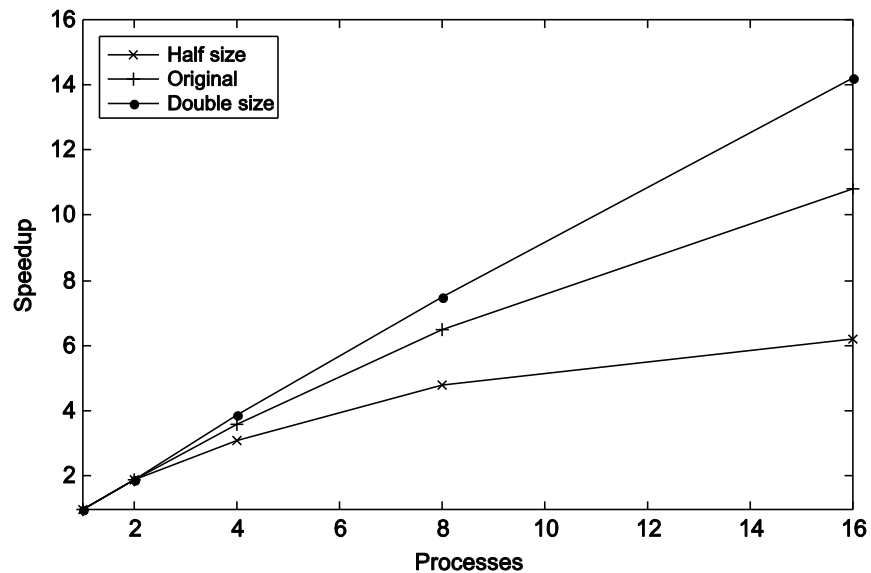
p	1	2	4	8	16
S	1.0	1.9	3.6	6.5	10.8
$E = S/p$	1.0	0.95	0.90	0.81	0.68

Performance on different problem sizes

- Performance metrics are determined not just by the number of processes (p), but significantly by the problem size.
- When we increase the problem size, the speedups and the efficiencies increase, while they decrease when we decrease the problem size. This behavior is quite common.

	p	1	2	4	8	16
Half	S	1.0	1.9	3.1	4.8	6.2
	E	1.0	0.95	0.78	0.60	0.39
Original	S	1.0	1.9	3.6	6.5	10.8
	E	1.0	0.95	0.90	0.81	0.68
Double	S	1.0	1.9	3.9	7.5	14.2
	E	1.0	0.95	0.98	0.94	0.89

Speedup vs Efficiency



Effect of Overhead

- Many parallel programs are developed by dividing the work of the serial program among the processes/threads and adding in the necessary “parallel overhead” such as mutual exclusion or communication.

$$T_p = T_s / p + T_o$$

Amdahl's Law (1967)

Unless virtually all of a serial program is parallelized, the possible speedup is going to be very limited — regardless of the number of cores available.

$$S_p = \frac{1}{f + (1 - f) / p}$$

where f is the fraction of the program that must be performed sequentially.

Example

- Suppose we can parallelize 90% of a serial program.
- If $T_s = 20$ seconds, runtime of parallelizable part is

$$0.9 \times T_s / p = 18 / p$$

- Runtime of “unparallelizable” part is

$$0.1 \times T_s = 2$$

- Overall parallel run-time is

$$T_p = 0.9 \times T_s / p + 0.1 \times T_s = 18 / p + 2$$

Example (cont.)

- Speed up

$$S_p = \frac{T_s}{0.9 \times T_s / p + 0.1 \times T_s} = \frac{20}{18 / p + 2}$$

That is, $S \leq 10$.

Parallelizing 90% of the program, and even with 1000 cores, we'll never get a speedup better than 10 !!!

In practice, ...

- There are several reasons not to be too worried by Amdahl's law.
- There are thousands of programs used by scientists and engineers that routinely obtain huge speedups on large distributed-memory systems.
- In many cases, obtaining a speedup of 5 or 10 is more than adequate, especially if the effort involved in developing the parallel program wasn't very large.

Gustafson's Law

- While Amdahl's Law often feels like a pessimistic view of parallel computing, focusing on the **limitations caused by serial bottlenecks**.
- Gustafson's Law (also known as Gustafson-Barsis's Law) provides a more optimistic perspective that better describes how we use HPC in the real world.
- While Amdahl's Law assumes a fixed problem size, Gustafson's Law assumes that as we gain more processing power, we use it to tackle **larger, more complex problems** in the same amount of time.

Scaled Speedup

- Users rarely run a small problem on a massive supercomputer just to finish it in milliseconds.
- Instead, use the extra cores to solve larger problems, e.g., increase the resolution of a simulation, use a larger dataset, or perform more iterations.
- In this model, the **execution time is kept constant**, but the amount of work performed increases – referred as *scaled speedup*.

Gustafson's Law

Speedup:

$$S_p = p + (1 - p) f$$

where p is the number of processes, and f is the non-parallelizable (serial) fraction of the process.

Example

Q: An application executing on 64 processes requires 220 seconds to run. Benchmarking reveals that 5% of the time is spent executing serial portions of the computation on a single processor. What is the scaled speedup of the application?

A:

$$\begin{aligned} S &= 64 + (1 - 64) \times (1 - 64)(0.05) \\ &= 64 - 3.15 = 60.85 \end{aligned}$$

Scaled Speedup

- Unlike Amdahl's Law, which assumes fixed problem size, Gustafson's Law assumes scaled workloads.
- As p increases, the parallel portion grows, allowing near-linear speedup when f is small.
- The serial fraction still limits performance, but far less restrictively than in Amdahl's model.

The Karp-Flatt Metric

- While the other two laws predict theoretical speedup based on a fixed or scaled workload.
- Karp-Flatt provides a way to measure the actual efficiency of a parallel program by looking at the **experimentally determined serial fraction**.
- Reveals overheads that Amdahl's Law often ignores, such as communication costs, synchronization delays, and thread creation time.

The Karp-Flatt Metric

Given a parallel computation exhibiting speedup S_p on p processes, the experimentally determined serial fraction e defined to be

$$e = \frac{1/S_p - 1/p}{1 - 1/p}$$

How to use the Karp-Flatt Metric

- If you are running a parallel algorithm on a cluster and you notice that speedup is not linear, Karp-Flatt helps you diagnose *why*.
 - The code inherently may have a lot of serial parts (Amdahl's Law).
 - The hardware or software stack, (e.g., MPI) is causing delays as you add more processors.

Interpreting the Karp-Flatt Metric

- Calculating e for different numbers of processors, you can identify the bottleneck
 - If e remains constant as you increase the number of processes (p), it suggests that the speedup is limited primarily by the serial fraction of the code.
 - If e increases as you increase p , it indicates that parallel overhead is growing, showing that the algorithm does not scale well with more hardware.

Example

Benchmarking a parallel program on 1, 2, ..., 8 processes produces the following speed-up results:

p	2	3	4	5	6	7	8
S	1.82	2.50	3.08	3.57	4.00	4.38	4.71

Q: What is the primary reason for the parallel program achieving a speedup of only 4.71 on eight processors?

Example, cont'd

p	2	3	4	5	6	7	8
S	1.82	2.50	3.08	3.57	4.00	4.38	4.71
e	0.10	0.10	0.10	0.10	0.10	0.10	0.10

A: Since the experimentally determined serial fraction is not increasing with the number of processors, the primary reason for the poor speedup is the limited opportunity for parallelism, that is, the large fraction of the computation that is inherently sequential.

Example

Benchmarking a parallel program on 1, 2, ..., 8 processes produces the following speed-up results:

p	2	3	4	5	6	7	8
S	1.87	2.61	3.23	3.73	4.14	4.46	4.71

Q:What is the primary reason for the parallel program achieving a speedup of only 4.71 on eight processors?

Example, cont'd

We begin by computing the experimentally determined serial fraction for each of these program runs:

p	2	3	4	5	6	7	8
S	1.87	2.61	3.23	3.73	4.14	4.46	4.71
e	0.070	0.075	0.080	0.085	0.090	0.095	0.1

A: Since the experimentally determined serial fraction is steadily increasing as the number of processors increases, the principal reason for the poor speedup is parallel overhead. This could be time spent in process startup, or communication, it could be an architectural constraint.

Interpreting the Karp-Flatt Metric

- Instead of guessing: “Is my algorithm inherently serial?”
- You can measure: “How much of my execution behaves like serial?”
- Helps distinguish:
 - algorithmic limits vs.
 - system/implementation inefficiencies.
- Amdahl’s Law states theoretical limit, but Karp–Flatt metric gives empirical diagnosis.

Taking Timings

- What is time?
- Start to finish?
- A program segment of interest?
- CPU time?
- Wall clock time?

Taking Timings

```
double start, finish;  
. . .  
start = Get_current_time();  
/* Code that we want to time */  
. . .  
finish = Get_current_time();  
printf("The elapsed time = %e seconds\n", finish-start);
```

Taking Timings

```
double global_elapsed;
double my_start, my_finish, my_elapsed;
. . .
/* Synchronize all processes/threads */
Barrier();
my_start = Get_current_time();

/* Code that we want to time */
. . .

my_finish = Get_current_time();
my_elapsed = my_finish - my_start;

/* Find the max across all processes/threads */
global_elapsed = Global_max(my_elapsed);
if (my_rank == 0)
    printf("The elapsed time = %e seconds\n", global_elapsed);
```

Scalability

- A problem (or system) is *scalable* if it can handle increasing problem sizes efficiently.
- This means that as the workload grows, the algorithm or system can still process it without a disproportionate increase in execution time.
- Scalability ensures that a system or algorithm can be extended to larger datasets or more demanding computations without becoming unusable.

Strong Scalability

- Efficiency remains constant as the number of processes increases while the problem size stays the same.
- Strong scalability shows how well a parallel system can speed up a fixed workload.
 - Measures the ability to reduce execution time by adding more processors.
- Often limited by Amdahl's Law, because the serial portion of the computation eventually dominates the total execution time.

Weak Scalability

- Efficiency remains constant as the number of processes increases by increasing the problem size proportionally.
- Reflects how a system handles growing workloads while maintaining efficiency.
 - Useful for large-scale simulations or data-intensive tasks, as it demonstrates the system's ability to handle bigger problems as more resources are added.
- Overheads from communication can still limit performance, so not all problems scale weakly.

In Practice ...

- Understanding strong vs weak scalability helps algorithm designers choose whether to optimize for fixed workloads (strong) or growing workloads (weak).
- When benchmarking a parallel system, reporting both types of scalability gives a fuller picture of its capabilities.
- Helps in deciding how many processors to allocate for a given problem size to maximize efficiency without wasting resources.

